“欧拉定理”的 Lean 4 形式化证明

——素数无穷性的机器验证

实验主题	对任意自然数 n，存在素数 p > n
验证工具	Lean 4.35.0 / 4.36.0-rc1；Mathlib v4.35.0 / v4.36.0-rc1
报告日期	2026 年 10 月 26 日

摘要：本实验以“对任意自然数 n，存在素数 p > n”为证明目标，使用 Lean 4 与 Mathlib 完成形式化验证。首先澄清题面名称与数学内容之间的差异：该命题按内容对应素数无穷性，通常称为欧几里得定理，而欧拉定理另有表述；题面括号中的整除命题只作为 Lean 输出格式的参考，不作为本实验的主要对象。随后给出基于 n! + 1 的经典反证证明，并把它拆解为“素数因子的存在性、阶乘的整除性、整除的加法消去”三个形式化步骤。最终代码以 `import Mathlib` 开头，不含 `sorry`、`admit` 或自定义公理，并以 `#print axioms` 完成审计。实验在 Lean 4.35.0 与 Lean 4.36.0-rc1 下均编译通过，公理输出仅包含 `propext`、`Classical.choice` 与 `Quot.sound`。报告也简要记录了文件编码导致的 `1:0` 解析错误及版本、缓存对齐问题，说明形式化证明的可复现性同时依赖数学论证与工具链环境。

关键词：Lean 4；Mathlib；形式化证明；素数无穷性；欧几里得定理；公理审计

一、实验背景与目标

形式化证明的价值在于把数学论证转化为可以被机器逐行检查的对象。自然语言证明中“显然”“同理”或“取一个素因子”等步骤，在证明助手中都必须落到具体定义、引理和推理规则上。Lean 4 以依赖类型论为基础，Mathlib 则提供了大量经过整理的数论、代数和分析结果。本实验选择素数无穷性作为对象，是因为它同时具备三个特点：命题陈述短、证明思想经典、形式化时需要处理若干具体的库引理与自然数运算细节。

题面要求证明：对任意自然数 n，存在素数 p > n。同时给出了 Lean 文件的格式与提交约束：文件以 `import Mathlib` 开头，定理声明要准确，不能使用 `sorry`、`admit` 或自行添加公理，并在末尾用 `#print axioms` 输出审计结果。由此，本实验的目标不只是“写出一段看起来像证明的文字”，而是交付一个能够被 Lean 内核接受的完整文件。

题面括号中的 `6 ∣ n³ − n` 是另一个独立的整数整除命题。它在本实验中的作用仅限于展示 Lean 文件的组织格式：导入 Mathlib、声明定理、给出证明、输出公理依赖。本报告不展开该整除命题的证明，只在需要说明格式时提及。

实验目标：（1）澄清待证命题的准确数学含义；（2）给出可读的数学证明；（3）把证明完整迁移到 Lean 4；（4）通过编译和公理审计确认没有隐藏假设；（5）总结一套可复用的证明 Agent 工作流。

二、发现问题

2.1 名称与内容的错位

实验开始时首先发现一个需要澄清的问题：题面把“对任意自然数 n，存在素数 p > n”称为“欧拉定律”，但这一命题的标准名称是**欧几里得定理**（Euclid's theorem），即素数有无穷多个。通常所说的欧拉定理是模算术中的 $a^{\phi(n)} \equiv 1 \pmod{n}$（当 gcd(a, n) = 1 时），与“素数无上界”不是同一个命题。

这一错位不是文字游戏，而是形式化工作的第一个风险点：如果直接按照“欧拉定理”这个名称去搜索库定理，可能找到完全不同的对象；如果按照括号中的整除例子去写文件，又会把另一个命题当成主命题。为避免混淆，本文在标题中保留题面称呼，但正文以形式化命题为准，称之为“素数无穷性命题”。题面括号中的整除命题只作为格式参考，不进入主证明。

2.2 验收条件与形式化边界

在动手写证明之前，需要把“完成”定义清楚。本实验采用四条验收条件：

1. **陈述准确**。目标必须是 `∃ p : ℕ, Nat.Prime p ∧ n < p`，不能把严格不等式改成 `n ≤ p`，也不能把它偷换成“素数集合有限”的否定而不加说明。
2. **证明完整**。不能出现 `sorry`、`admit`，也不能用 `axiom` 或 `constant` 把目标直接假设掉。
3. **公理可审计**。末尾用 `#print axioms` 输出依赖；允许 Lean/Mathlib 的标准基础公理，但不允许 `sorryAx` 或自定义公理。
4. **环境可复现**。Lean 与 Mathlib 的版本必须匹配，缓存必须就位，文件编码不能引入不可见前缀。

2.3 工具链与文件层面的异常

开发过程中曾出现 `Challenge.lean:1:0: expected token`。这个报错位置在第 1 行第 0 列，说明 Lean 在读取文件的第一个 token 时就失败了，而不是某个证明步骤失败。经复现，文件开头若含 UTF-8 BOM、零宽字符或中文说明，都会得到同样的错误；`import Mathlib` 本身不会触发它。把文件保存为 UTF-8 无 BOM、确保首行从第 1 列开始，问题即消失。

另一个工程问题是版本与缓存：Lean 4.35 与 4.36 的 Mathlib 构建产物不能随意混用；没有 Mathlib 缓存时，`import Mathlib` 会报找不到模块或找不到 `.olean`，而不是 `1:0`。因此，形式化证明的“运行成功”至少包含三个层面：数学证明正确、Lean 语法与类型正确、工具链环境匹配。本报告的重点仍是第一项，但后两项决定了结果能否被复现。

三、分析问题

3.1 数学证明

设 n 是任意自然数，令

$$N = n! + 1。$$

因为 $n! \geqslant 1$，所以 $N \geqslant 2$，特别地 $N \neq 1$。由“每个不等于 1 的自然数都有素因子”这一基本事实，存在素数 p 使得 $p \mid N$。下面证明 $p > n$。

反设 $p \leqslant n$。由于 p 是素数，$p > 0$；而 $p \leqslant n$，所以 p 是 $1, 2, \cdots, n$ 中的一个因子，于是 $p \mid n!$。另一方面，由 p 的选取有 $p \mid n! + 1$。两个整除关系同时成立，便得到

$$p \mid (n! + 1) - n! = 1。$$

但素数 p 不可能整除 1，矛盾。因此反设不成立，必有 $p > n$。

由于 n 是任意的，上述结论说明素数没有上界：对任何自然数，都能找到一个比它更大的素数。这正是“素数有无穷多个”的常见表述。证明的关键不是显式构造一个素数序列，而是利用 $n! + 1$ 把所有不超过 n 的候选因子“排除”掉：任何不超过 n 的素数都整除 $n!$，不可能再整除 $n! + 1$。

3.2 从数学步骤到 Lean 引理

自然语言证明中的每一步，都需要在 Mathlib 中找到对应的表达。表 1 给出了本实验使用的映射关系。

表 1　数学步骤与 Lean 表达的对应

数学陈述	Lean 表达	作用
n 是自然数	`n : ℕ`	固定论域
$n! + 1$	`n.factorial + 1`	构造关键整数 N
$N \neq 1$，故有素因子	`Nat.exists_prime_and_dvd (n := n.factorial + 1)`	得到 p、p 为素数、$p \mid N$
p 为素数，故 $p > 0$	`hp.pos`	满足阶乘整除性的正性条件
$p \leqslant n$，故 $p \mid n!$	`Nat.dvd_factorial hp.pos hle`	把“在阶乘范围内”转成整除
$p \mid n!$ 且 $p \mid n! + 1$，故 $p \mid 1$	`(Nat.dvd_add_iff_right hfac).2 hpd`	整除的加法消去
素数不整除 1	`hp.not_dvd_one`	导出矛盾
由 $\neg(n < p)$ 得 $p \leqslant n$	`by omega`	自然数线性算术

这里有两个容易出错的细节。第一，库引理 `Nat.dvd_add_iff_right` 的方向是“若 $p \mid m$，则 $p \mid n \leftrightarrow p \mid m + n$”。本证明中取 $m = n!$、$n = 1$，因此用 `.2` 把 $p \mid n! + 1$ 转成 $p \mid 1$。第二，阶乘整除引理要求 $p > 0$；如果忽略素数的正性条件，类型检查不会通过。两者都不是“显然”可以跳过的步骤，而是形式化必须显式交代的细节。

3.3 证明骨架

在 Lean 中，证明可以组织成如下骨架。外层先取素因子，内层用反证法处理严格不等式；矛盾由“素数整除 1”关闭。

```
目标：∃ p : ℕ, Nat.Prime p ∧ n < p
├─ 取 p，满足 hp : Nat.Prime p，hpd : p ∣ n! + 1
├─ 反设 ¬ n < p，由 omega 得 hle : p ≤ n
├─ 由 hle 得 hfac : p ∣ n!（Nat.dvd_factorial）
├─ 由 hfac、hpd 得 p ∣ 1（Nat.dvd_add_iff_right）
└─ 与 hp.not_dvd_one 矛盾，故 n < p
```

把骨架写成 Lean 时，还要处理一个“顺序”问题：`Nat.exists_prime_and_dvd` 返回的是 `∃ p, Prime p ∧ p ∣ N`，而目标要求的是 `∃ p, Prime p ∧ n < p`。因此需要先用 `obtain` 取出见证，再用 `refine ⟨p, hp, ?_⟩` 重新组装目标，把剩下的严格不等式留作子目标。这种“取见证—重排结论—关闭剩余目标”的模式，是 Lean 中处理存在量词证明的常见结构。

3.4 难点分析

- **严格不等式的方向**。反证法给出的是 `¬ n < p`，需要转成 `p ≤ n`；这一步用 `omega` 比手写自然数次序引理更稳定。
- **自然数没有减法**。数学上写成 (n! + 1) − n! = 1，但在 Lean 中使用 `Nat.dvd_add_iff_right` 更直接，避免截断减法和附加的 ⩽ 条件。
- **库引理的签名**。`Nat.dvd_factorial` 需要正性和 ⩽ 两个前提，`Nat.Prime.pos` 与 `omega` 分别提供它们；凭记忆写 API 很容易在这里失败。
- **避免循环论证**。Mathlib 已有 `Nat.exists_infinite_primes`，它几乎就是本题的结论。若直接调用它，形式上固然能通过，但会把待证命题当作前提。本实验只使用更基本的“每个大于 1 的自然数有素因子”引理，保持证明的非循环性。
- **环境与编码**。版本不匹配、缓存缺失、BOM 前缀都会造成编译失败，但它们与数学证明无关，需要与证明错误分开排查。

四、解决问题

4.1 Agent 工作流

为了让证明过程可复用，本实验把工作拆成一个五阶段的 Agent 工作流，而不是一次性生成整段代码。表 2 概括了每个阶段的输入、动作和输出。

表 2　证明 Agent 的工作流与质量门禁

阶段	动作	输出/门禁
冻结声明	把自然语言命题翻译成精确的 Lean 类型，确认论域、量词和严格不等式。	签名与题面一致；不擅自加假设。
检索库	用 `rg`、`#check` 查 Mathlib 中的素因子、阶乘、整除引理；以实际签名而非记忆为准。	列出候选引理及前提条件。
搭建骨架	先写 `obtain`、`refine`、`by_contra`，把大目标拆成可独立编译的 `have`。	每个中间步骤有明确类型。
编译修复	运行 `lake env lean`；先读第一条错误；每次只改一小块。	退出码为 0，无未解决目标。
审计交付	检查首行、禁用词、定理签名，运行 `#print axioms`。	无 `sorryAx`、无自定义公理。

这套流程的关键是“最小修复”和“可审计”。前者避免一次改太多导致错误互相掩盖；后者把“看起来证完了”变成可检查的客观条件。对本题而言，真正需要人判断的地方只有两处：选择 n! + 1 这个构造，以及决定使用哪些库引理。其余步骤都可以由编译器给出明确的成功或失败信号。

4.2 Lean 实现

完整代码如下。文件首行为 `import Mathlib`，末尾用 `#print axioms` 输出审计结果。

```
import Mathlib

theorem exists_prime_gt (n : ℕ) : ∃ p : ℕ, Nat.Prime p ∧ n < p := by
  obtain ⟨p, hp, hpd⟩ := Nat.exists_prime_and_dvd (n := n.factorial + 1) (by
    have := Nat.factorial_pos n
    omega)
  refine ⟨p, hp, ?_⟩
  by_contra h
  have hle : p ≤ n := by omega
  have hfac : p ∣ n.factorial := Nat.dvd_factorial hp.pos hle
  exact hp.not_dvd_one ((Nat.dvd_add_iff_right hfac).2 hpd)

#print axioms exists_prime_gt
```

4.3 代码解释

第一行导入 Mathlib，随后声明目标定理。证明主体从 `obtain` 开始：它把 `Nat.exists_prime_and_dvd` 的存在性结论拆成三项——见证 `p`、素性 `hp`、整除性 `hpd`。这里给 `n` 参数显式传入 `n.factorial + 1`，并用 `have := Nat.factorial_pos n` 与 `omega` 证明 `n.factorial + 1 ≠ 1`，满足库引理的前提。

接着 `refine ⟨p, hp, ?_⟩` 把目标重排为“`p` 是素数且 `n < p`”，其中前两项已经由 `hp` 提供，剩下的问号就是严格不等式。随后进入反证：`by_contra h` 假设 `¬ n < p`，`omega` 将其转成 `p ≤ n`。有了正性 `hp.pos` 和上界 `hle`，`Nat.dvd_factorial` 给出 `p ∣ n.factorial`。

最关键的一步是 `(Nat.dvd_add_iff_right hfac).2 hpd`：`hfac` 说明 `p` 整除 `n!`，因此“`p` 整除 1”与“`p` 整除 `n! + 1`”等价；`hpd` 正好是后者的证明，于是得到 `p ∣ 1`。最后 `hp.not_dvd_one` 指出素数不可能整除 1，矛盾完成。整个过程没有使用 `sorry`、`admit` 或自定义公理，也没有调用 `Nat.exists_infinite_primes` 这个“现成结论”。

4.4 验证结果

证明分别在两条工具链上编译。表 3 汇总了版本、命令和结果。

表 3　编译与公理审计结果

环境	命令	结果
Lean 4.35.0 Mathlib v4.35.0	`lake build`	成功；共 9050 个构建任务
Lean 4.36.0-rc1 Mathlib v4.36.0-rc1	`lake build`	成功；共 18114 个构建任务

两次构建都给出了相同的公理审计结果：

```
'exists_prime_gt' depends on axioms: [propext, Classical.choice, Quot.sound]
```

这三个名字是 Lean 与 Mathlib 的标准基础公理，不是本证明自行声明的公理。`propext` 是命题外延性，`Classical.choice` 是经典选择，`Quot.sound` 是商类型的相等规则；它们属于 Lean 的逻辑基础。输出中没有 `sorryAx`，也没有任何用户自定义公理。对源文件做禁用词扫描，同样没有发现 `sorry`、`admit`、`axiom`、`constant` 或 `native_decide`。

在 Lean 4.36 下，非模块文件导入 Mathlib 会产生一条“建议添加 `module`”的警告。由于题面要求首行必须是 `import Mathlib`，本实验保留非模块写法；该警告不影响编译结果，也不影响定理的公理审计。

4.5 环境问题的处理

针对前文提到的 `1:0` 报错，处理方式是：用 VSCode 右下角的编码菜单确认文件为 UTF-8（不是 UTF-8 with BOM）；必要时把文件开头的中文说明、Markdown 围栏或零宽字符删除，使首字节直接是 `import` 的 `i`。针对缓存问题，先在项目根目录运行 `lake exe cache get` 下载 Mathlib 预编译产物，再运行 `lake build`；直接编译源码在时间上不可行。针对版本问题，Lean 与 Mathlib 必须整套对齐：4.35.0 配 v4.35.0，4.36.0-rc1 配 v4.36.0-rc1。

这些处理没有改变定理陈述，也没有引入任何额外假设。它们只是让编译器能够在正确的环境中读取同一份证明。这一点对实验报告很重要：形式化失败有两种完全不同的原因——证明不成立，或证明没有被正确送入检查器；必须先用最小错误定位区分二者，再决定是改证明还是改环境。

五、结果与讨论

5.1 实验结论

本实验完成了素数无穷性命题的 Lean 4 形式化证明。定理 `exists_prime_gt` 的类型为

$$\forall\, n : \mathbb{N},\ \exists\, p : \mathbb{N},\ \mathrm{Nat.Prime}\ p \wedge n < p,$$

证明不含 `sorry`、`admit` 或自定义公理；在 Lean 4.35.0 和 4.36.0-rc1 下均编译成功；`#print axioms` 只报告 `propext`、`Classical.choice`、`Quot.sound` 三项标准基础。就形式化验证的意义而言，目标已经达成。

5.2 形式化与自然语言证明的差异

自然语言证明可以一句话带过“$n! + 1$ 有素因子”，但在 Lean 中必须调用一个明确的库引理，并证明它的前提 $n! + 1 \neq 1$。自然语言可以写“于是 p 整除 1”，但 Lean 需要说明这是从 $p \mid n!$ 和 $p \mid n! + 1$ 通过哪条加法消去规则得到的。形式化的额外工作，主要就集中在这些“显然”的接缝处。

另一方面，形式化也带来了自然语言没有的确定性：`#print axioms` 把“没有偷偷加假设”变成可检查的输出；编译器把“证明是否完整”变成退出码；禁用词扫描把“没有用 `sorry`”变成可复核的事实。对于教学作业而言，这种可审计性比单纯的文字证明更有价值。

5.3 局限与改进

本证明依赖 Mathlib 的素因子存在性引理。该引理本身是数论的基本事实，不是本题的结论，因此不构成循环论证；但如果要求“从更原始的公理出发”，还需要进一步展开 `Nat.minFac` 的实现与正确性。另一方面，本证明证明的是“无上界”形式，而不是显式构造一个素数序列；两者在经典逻辑下等价，但后者需要更多构造性内容。

工程上还可以改进两点：其一，把版本固定、缓存检查和编码检查写进脚本，减少环境导致的失败；其二，把证明拆成独立的辅助引理（例如“每个大于 1 的自然数有素因子”与“素数不整除 1”），使错误定位更细。当前的单文件形式适合作业提交，但不利于大规模复用。

题面括号中的整除命题在本实验中只承担“格式示例”的角色：它提醒我们 Lean 文件应如何组织，以及 `#print axioms` 应放在哪里。它的数学内容没有被纳入主证明，也没有影响素数无穷性论证的结构。

六、总结

本次实验按照“发现问题—分析问题—解决问题”的路径完成。发现问题阶段，澄清了“欧拉定理”名称与“素数无穷性”内容之间的错位，并把整除示例限定为格式参考；分析问题阶段，把经典反证证明拆成素因子存在、阶乘整除、加法消去三个可形式化的步骤；解决问题阶段，写出一份不含 `sorry` 与自定义公理、能够被 Lean 4.35/4.36 编译的证明，并用 `#print axioms` 完成审计。

实验的直接结论是：对任意自然数 n，确实存在素数 p > n；机器检查确认了这一点。更一般的收获是：形式化证明的难点往往不在宏大的数学思想，而在把“显然”的步骤拆成编译器能够接受的引理应用；而一个可靠的证明 Agent，应当同时管理数学论证、库 API、编译环境和审计证据，而不是只生成一段漂亮的代码。

参考文献

[1] Lean 4 官方文档. https://lean-lang.org/

[2] The Mathlib Community. Mathlib4. https://github.com/leanprover-community/mathlib4

[3] Euclid. Elements, Book IX, Proposition 20（素数无穷性）.

[4] Avigad J., de Moura L., Kong S., et al. Theorem Proving in Lean 4. https://leanprover.github.io/theorem_proving_in_lean4/

附录 A　复现命令

```
cd lean-proof-agent
lake update
lake exe cache get
lake build
lake env lean Challenge.lean
```

说明：`lake exe cache get` 用于下载 Mathlib 预编译缓存；版本必须与 `lean-toolchain` 及 `lakefile.toml` 中的 Mathlib 版本一致。

附录 B　验证记录

```
Lean 4.35.0 + Mathlib v4.35.0
Build completed successfully (9050 jobs).

Lean 4.36.0-rc1 + Mathlib v4.36.0-rc1
Build completed successfully (18114 jobs).

'exists_prime_gt' depends on axioms: [propext, Classical.choice, Quot.sound]

禁用词扫描：无 sorry / admit / axiom / constant / native_decide / unsafe / extern
源文件 SHA-256：3abdbb563b51f22a005a2c4f3c4ea651956c5db4d057ca38b046c690b2649cd0
```

附录 C　公理说明

公理审计输出的三个名字均来自 Lean 的逻辑基础，而不是本题新增的假设。表 C1 给出简要说明。

表 C1　审计输出中的标准基础公理

名称	含义	来源
`propext`	命题外延性：若两个命题互相蕴含，则把它们视为相等。	Lean 逻辑基础
`Classical.choice`	经典选择：从存在性命题中取出一组见证。	经典逻辑
`Quot.sound`	商类型中，被商关系关联的两个元素在商中相等。	商类型构造

本题没有使用 `axiom`、`constant` 或 `sorry` 去声明新的假设，因此审计输出中不会出现自定义公理名。若在输出中看到 `sorryAx`，则说明证明仍有未完成的目标；若看到项目自定义的名字，则说明有人为引入了额外假设。这两种情况都不属于本实验的合格结果。

从可复现性角度看，公理审计只覆盖“证明依赖了哪些逻辑原则”，不覆盖环境是否正确。一个证明可能在 4.35 下通过、在 4.36 下因 API 或缓存问题失败；因此本实验把“编译成功”和“公理审计”作为两个独立的验收条件，并在附录 B 中同时记录。